

Lost Map

Day 4 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“reconstruct the map given only the table”*
- ▶ *“output $n-1$ lines with two integers”*

Programming interpretation:

- ▶ The distance table describes all-pairs shortest-path distances from an original tree.
- ▶ The original roads are recovered by selecting a minimum spanning tree.
- ▶ Kruskal's algorithm is a direct fit for building that tree from weighted edges.

Textbook connection: CP4 4.3.2 Kruskal's Algorithm

Core technique: Minimum spanning tree with union-find

- ▶ Villages are vertices.
- ▶ The distance matrix gives the weight of every possible road.
- ▶ We need roads that connect all villages with minimum total cost.

- ▶ Connecting all vertices at minimum weight is an MST problem.
- ▶ The input is dense, so converting the matrix to an edge list is straightforward.
- ▶ Kruskal's algorithm matches the edge-list representation.

- ▶ Read the distance matrix.
- ▶ Create an edge list of (weight, u, v) pairs.
- ▶ Sort edges by increasing weight.
- ▶ Use union-find to add an edge only when it connects two different components.
- ▶ Stop after selecting $N - 1$ edges and print their endpoints.

- ▶ The solution stores edges and pops them from smallest to largest.
- ▶ Union-find prevents cycles in the selected road set.
- ▶ The final list has exactly enough edges to connect all villages.

- ▶ Kruskal's cut property says the lightest safe edge can be added to some MST.
- ▶ Union-find identifies whether an edge is safe by detecting if it would form a cycle.
- ▶ After $N - 1$ safe edges, the selected graph is connected and acyclic, so it is an MST.

Complexity: $O(E \log E)$ time and $O(E + V)$ memory; for this complete graph E is $O(V^2)$.

- ▶ Output uses 1-based village numbers.
- ▶ The diagonal self-edges are harmless if ignored by union-find, but they are not useful MST edges.

Source lines: `lostmap.py:53-82`

```
class UnionFind:

    def __init__(self, numItems: int) -> None:
        if numItems < 1:
            raise ValueError("numItems needs to be greater or equal to 1")

        self._numSet = numItems
        self._parents = list(range(numItems))
        self._ranks = [0] * numItems
        self._sizes = [1] * numItems
```

What this chunk does

- ▶ The MST builder needs a DSU to avoid cycles.
- ▶ Every village begins in its own component.
- ▶ Rank and size support efficient merging.

Source lines: `lostmap.py:84-115`

```
def findSet(self, element: int) -> int:
    children = []
    toFind = element

    while toFind != self._parents[toFind]:
        children.append(toFind)
        toFind = self._parents[toFind]

    for c in children:
        self._parents[c] = toFind

    return toFind
```

What this chunk does

- ▶ `findSet` walks to the representative of a component.
- ▶ The visited nodes are compressed to point directly to the representative.
- ▶ This keeps repeated Kruskal checks fast.

Source lines: `lostmap.py:117-155`

```
def isSameSet(self, element1: int, element2: int) -> bool:
    return self.findSet(element1) == self.findSet(element2)

def unionSet(self, element1: int, element2: int) -> None:
    s1 = self.findSet(element1)
    s2 = self.findSet(element2)

    if s1 == s2:
        return None

    if self._ranks[s1] > self._ranks[s2]:
        s1, s2 = s2, s1
    elif self._ranks[s1] == self._ranks[s2]:
        self._ranks[s2] += 1

    self._parents[s1] = s2

    self._numSet -= 1
    self._sizes[s2] += self._sizes[s1]
```

What this chunk does

- ▶ `isSameSet` checks whether an edge would create a cycle.
- ▶ `unionSet` links two different components by rank.
- ▶ Component sizes and the component count are updated after a successful merge.

Source lines: `lostmap.py:177-205`

```
def kruskal(
    numVertex: int, edgeList: list[tuple[int, int, int]]
) -> tuple[list[tuple[int, int, int]], int]:
    edges = sorted(edgeList, key=lambda x: -x[0])
    udfs = UnionFind(numVertex)
    edgeListOutput = []
    totalMSTweight = 0
    while edges and len(edgeListOutput) < numVertex - 1:
        w, u, v = edges.pop()
        ru = udfs.findSet(u)
        rv = udfs.findSet(v)
        if ru != rv:
            udfs.unionSet(ru, rv)
            edgeListOutput.append((w, u, v))
            totalMSTweight += w
    return (edgeListOutput, totalMSTweight)
```

What this chunk does

- ▶ All candidate roads are sorted by weight.
- ▶ The smallest remaining edge is accepted only if it joins different components.
- ▶ The loop stops after $n - 1$ edges, exactly the size of a spanning tree.

Source lines: lostmap.py:208-218

```
lines = sys.stdin.read().strip().splitlines()
grid = [list(map(int, line.split())) for line in lines[1:]]
numVertex = int(lines[0])
output = []
edgeList = []
for row in range(numVertex):
    for col in range(row, numVertex):
        edgeList.append((grid[row][col], row, col))
for edge in kruskal(numVertex, edgeList)[0]:
    output.append(f"{edge[1]+1} {edge[2]+1}\n")
sys.stdout.write("".join(output))
```

What this chunk does

- ▶ The matrix is converted into weighted edges between villages.
- ▶ Only $\text{row} \leq \text{col}$ is used to avoid duplicating undirected edges.
- ▶ Kattis expects village numbers starting from 1, so output adds 1.

Solution file: `practice_contests/day_4/lostmap.py`

Problem: <https://open.kattis.com/problems/lostmap>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.